技术笔记 · LINUX KERNEL

# Linux 实时抢占

## PREEMPT_RT 原理、构建与调优

从抢占模型、内核改造、中断时序，到内核构建、系统调优、应用编程准则与延迟度量的一站式梳理。

**适用内核** Linux 6.12 及以上（PREEMPT_RT 已并入主线）

**文档版本** v1.0 **整理日期** 2026-11-08

# 目录


**1** 实时性的判据

1.1 实时不等于快　1.2 四类延迟　1.3 为什么平均值没有意义

**2** Linux 抢占模型的演进

2.1 内核态为何默认不可抢占　2.2 四种模型　2.3 PREEMPT_DYNAMIC　2.4 常见误解

**3** PREEMPT_RT 的核心改造

3.1 总体思路　3.2 五处改造　3.3 优先级继承　3.4 对驱动作者的约束

**4** 一次中断的端到端时序

4.1 时序对比　4.2 剩余不可控延迟源

**5** 获取与构建 RT 内核

5.1 三条途径　5.2 关键配置项　5.3 构建步骤

**6** 系统级调优

6.1 CPU 隔离　6.2 中断亲和性　6.3 电源管理　6.4 调度与内存

**7** 应用程序编写准则

7.1 优先级设计　7.2 实时循环禁忌　7.3 锁与优先级反转　7.4 代码骨架

**8** 度量与验证

8.1 cyclictest　8.2 ftrace 追踪器　8.3 参考量级与判读

**9** 代价、边界与替代方案

**10** 落地示例：RK3588 上的取舍

**A** 附录：术语表与命令速查

# 1 实时性的判据

PREEMPT_RT 的目标不是让 Linux 跑得更快，而是让**最坏情况下的响应时间变短、并且有界。**判断一个系统是否"实时"，只看一句话：从外部事件发生，到系统给出确定的响应，这段时间的上界能不能写进规格书并被反复验证。

## 1.1 实时不等于快

一个平均响应 50 微秒、但偶尔抖到 20 毫秒的系统，是**不可用**的实时系统；一个稳定在 200 微秒、最坏不超过 250 微秒的系统，才是合格的实时系统。工程上真正要交付的是**确定性**（determinism），而不是速度。

## 1.2 四类延迟

讨论实时性时必须先分清在说哪一段延迟，否则测出来的数字没法对齐：

| 延迟类型 | 起点 | 终点 | 主要来源 |
| --- | --- | --- | --- |
| 中断延迟 | 硬件中断引脚拉高 | 中断服务程序第一条指令 | 关中断区间、中断被屏蔽、CPU 在深睡眠 |
| 抢占延迟 | 高优先级任务被唤醒 | 内核决定切换 | `preempt_count` 非零、自旋锁、原子上下文 |
| 调度延迟 | 高优先级任务被唤醒 | 该任务真正开始执行 | 上述两者之和 + 上下文切换开销 |
| 最坏响应时间 | 外部事件 | 系统输出动作 | 端到端路径上所有环节的最坏值累加 |

## 1.3 为什么平均值没有意义

延迟的分布是长尾的：99% 的样本可能都在 50 微秒以内，剩下 1% 落在 5 毫秒。控制系统的稳定性由**尾部**决定——一个 5 毫秒的抖动就可能让电机环路过冲、让音频出现爆音、让采集数据错位。所以度量 RT 系统的指标永远是 **Max 与 P99.99**，而不是 Avg。

# 2 Linux 抢占模型的演进

## 2.1 内核态为何默认不可抢占

Linux 诞生时的设计取向是吞吐优先，内核态代码默认不可抢占，原因有三：

- **per-CPU 数据的无锁假设。**内核大量代码默认"当前 CPU 上只有我在跑"，一旦被抢占，另一个任务改了同一份 per-CPU 变量就会错乱。
- **临界区语义依赖关抢占。**`spin_lock_irqsave()` 一拿就是"关中断 + 关抢占"，期间任何任务都叫不动这个核。
- **吞吐代价。**处处可抢占意味着更多的 `preempt_count` 记账、更多的上下文切换与 cache 抖动。

上游的折中办法是"**在安全点主动让出**"——这就是 voluntary 模型。问题在于让不让出、让多快，完全取决于写那段代码的人，实时性无法承诺。

## 2.2 四种抢占模型




图 1　Linux 的四种抢占模型：从吞吐优先到确定性优先

| 模型 | 内核配置 | 典型用途 | 最坏延迟量级 |
| --- | --- | --- | --- |
| PREEMPT_NONE | `CONFIG_PREEMPT_NONE` | 服务器、大数据、批处理 | 数十 ms |
| PREEMPT_VOLUNTARY | `CONFIG_PREEMPT_VOLUNTARY` | 通用桌面、多数发行版默认 | 数 ms ~ 数十 ms |
| PREEMPT | `CONFIG_PREEMPT` | 桌面交互、软实时、Ubuntu lowlatency | 数百 µs ~ 数 ms |
| PREEMPT_RT | `CONFIG_PREEMPT_RT` | 工业控制、机器人、音频、车载 | 数十 µs ~ 数百 µs |

## 2.3 PREEMPT_DYNAMIC：运行时可切换

5.12 起内核提供 `CONFIG_PREEMPT_DYNAMIC`，允许在**启动参数**里选择抢占模型：

```
preempt=none | voluntary | full
```

Ubuntu 22.04 的 generic 内核就编译了该选项，默认取 voluntary。注意：切到 `full` 只是得到第三种模型（PREEMPT），**不是 RT**——它没有线程化中断、没有睡眠自旋锁、没有优先级继承。

### 2.4 常见误解

> **注意**
>
> **lowlatency ≠ realtime。**Ubuntu 的 `linux-lowlatency` 是基于 PREEMPT 模型做的调优版本，能改善桌面交互和音频延迟，但不提供硬实时保证。真正的 RT 内核必须来自 `CONFIG_PREEMPT_RT`。

## 3 PREEMPT_RT 的核心改造

### 3.1 总体思路

一句话：**把"不能被打断的东西"尽可能改造成"可以被打断的普通调度实体"。**凡是会让 CPU 停留在不可抢占状态的机制，都被替换成"可以被更高优先级任务抢占"的等价实现；剩下的极小部分用 `raw_spinlock_t` 明确标注为真正的原子操作。

### 3.2 五处改造

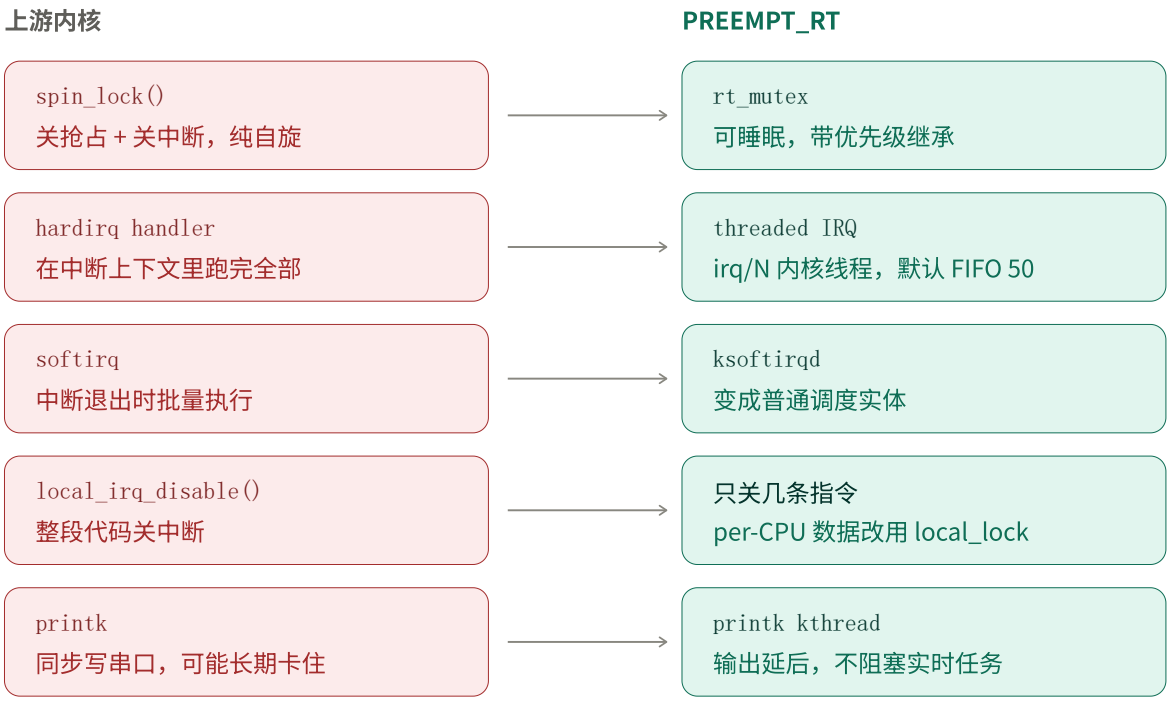



图 2　PREEMPT_RT 对五类内核机制的等价替换

**（1）自旋锁变成睡眠锁。**`spinlock_t` 在 RT 下由 `rt_mutex` 实现，`spin_lock()` 在竞争时会睡眠等待，并把等待者按优先级排队。真正需要"绝对不睡眠"的地方（如中断控制器、调度器内部）改用 `raw_spinlock_t`。

**（2）硬中断线程化。**除少数标记为 `IRQF_NO_THREAD` 的中断外，所有中断处理程序都运行在名为 `irq/N-设备名` 的内核线程里，默认 `SCHED_FIFO` 优先级 50，可被更高优先级的实时任务抢占。

**（3）软中断线程化。**绝大多数 softirq 交给 `ksoftirqd` 处理，只有极少数（如 RCU、定时器）保留在硬中断上下文直接处理。

**（4）关中断区间压缩。** `local_irq_disable()` 被限制到极短的几条指令；保护 per-CPU 数据改用 `local_lock_t`——它只禁止迁移与本地并发，不禁止抢占。

**（5）printk 异步化。**控制台输出交给 `printk` 线程，避免一条串口打印把实时任务卡住几百微秒到几毫秒。

## 3.3 优先级继承

RT 的锁带**优先级继承**（Priority Inheritance, PI）：当高优先级任务阻塞在一个被低优先级任务持有的锁上时，持锁者临时被提升到等待者的优先级，从而不会被中间优先级的任务抢占。




图 3　优先级反转：RT 用优先级继承把 L 临时提升到 80，使 M 无法抢占

PI 对用户空间同样生效：用 `PTHREAD_PRIO_INHERIT` 属性创建的互斥量，内核会通过 PI futex 完成优先级继承。**实时任务之间共享的锁必须带这个属性**，否则一次优先级反转就能让最坏延迟失去上界。

## 3.4 对驱动作者的约束

RT 改变了内核 API 的语义，写驱动/内核模块时必须遵守：

- `spin_lock()` 在 RT 下可能睡眠，**绝不能**在中断上下文、`preempt_disable()` 区间、或任何原子上下文调用它——会触发 `BUG: scheduling while atomic`。
- 确实需要原子语义的地方用 `raw_spinlock_t`，并保证临界区只有几条指令。
- 保护 per-CPU 数据用 `local_lock_t`，不要用 `preempt_disable()`。
- 只想禁止迁移、不禁止抢占的地方用 `migrate_disable()` / `migrate_enable()`。
- 中断处理里只做"取数据 + 唤醒"，耗时工作放到线程化部分。

# 4　一次中断的端到端时序

## 4.1 时序对比

把延迟拆到时间轴上，比记概念更直观：

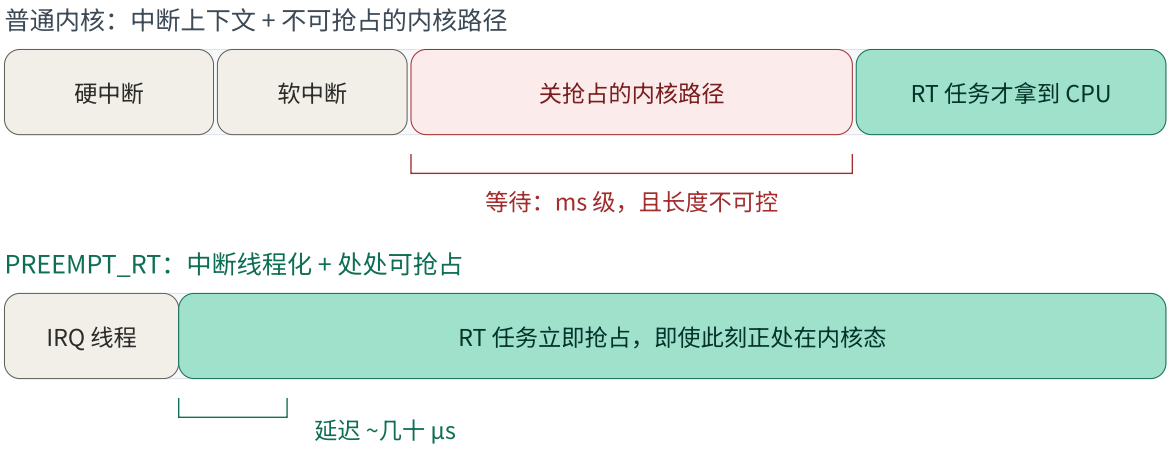



图 4　同一个中断在两种内核下的响应时序

## 4.2 剩余不可控延迟源

RT 内核消灭的是"内核自己制造的延迟"，下面这些它管不了——而它们往往才是实测超标的原因：

| 来源 | 机理 | 可采取的规避手段 |
| --- | --- | --- |
| 固件 / SMI | x86 的 SMI 会整体偷走 CPU 数百 µs，Linux 完全不知情；ARM 板上的 ATF 固件同理 | BIOS 里关闭不必要的 SMM 服务；选 SMI 行为已知的板卡 |
| DDR 刷新 / DMA 争用 | 内存控制器被 GPU、VPU、网卡 DMA 抢占，A76 也会等总线 | 实时任务绑核并避开高带宽外设；必要时降低外设带宽 |
| 深 C-state / DVFS | 从 C6 唤醒要几十 µs；频率切换会造成执行时间波动 | `intel_idle.max_cstate=0`、governor 固定 performance |
| cache / TLB miss | 首次访问的代码和数据总是慢的 | 启动时预热代码路径与数据；避免大工作集 |
| 页错误 | 缺页会触发内核分配与 IO，动辄数十 µs 到 ms | `mlockall()` 锁定内存；预分配所有缓冲区 |
| 透明大页合并 | `khugepaged` 后台合并会抢 CPU、造成抖动 | 关闭 THP 或设置 `madvise` 模式 |
| 虚拟化 | vCPU 被 hypervisor 调度出去，guest 无从感知 | 实时系统不要跑在 VM 里；必须跑就用 CPU 直通 + 独占核 |
| 热降频 | 温度触顶后频率骤降，执行时间翻倍 | 加强散热；监控 `thermal_zone`；留出算力余量 |

# 5　获取与构建 RT 内核

## 5.1 三条途径

| 途径 | 做法 | 适用 |
| --- | --- | --- |
| 主线 6.12+ | PREEMPT_RT 已于 6.12 合并进 mainline，直接 `CONFIG_PREEMPT_RT=y`，无需打补丁 | 新项目首选，维护成本最低 |
| 老内核补丁 | 从 kernel.org 的 `-rt` 分支取对应版本的 `patch-<ver>-rt<n>.patch`，版本号必须严格匹配 | 必须留在厂商 BSP 版本上时 |
| 发行版软件包 | Debian：`linux-image-rt-amd64`；Ubuntu 24.04+：`linux-realtime` | 快速验证、非定制硬件 |
| 芯片厂商 BSP | 向 FAE 索取配套 RT 补丁或分支（如 Rockchip 5.10 系列） | 需要厂商外设驱动时 |

> **注意**
>
> Ubuntu 22.04 官方仓库只有 `linux-lowlatency`，属于 PREEMPT 模型；官方 realtime 内核自 24.04 起提供。若必须在 22.04 上用 RT，请自行编译或使用 Ubuntu Pro 的 real-time 变体。

## 5.2 关键配置项

| 配置项 | 建议值 | 说明 |
| --- | --- | --- |
| `CONFIG_PREEMPT_RT` | y | 总开关，自动带上 `RT_MUTEXES`、`PREEMPT_COUNT` |
| `CONFIG_HIGH_RES_TIMERS` | y | 高精度定时器，RT 的前提 |
| `CONFIG_NO_HZ_FULL` | y | 配合 `nohz_full=` 让隔离核停掉 tick |
| `CONFIG_RT_GROUP_SCHED` | n | 开启后 RT 带宽按 cgroup 分配，容易误伤实时任务 |
| `CONFIG_DEBUG_PREEMPT` | 调试期 y | 检测原子上下文里的非法睡眠，正式版可关 |
| `CONFIG_PREEMPT_TRACER` | 调试期 y | 配合 ftrace 定位最长不可抢占区间 |
| `CONFIG_HZ` | 1000（或 250 + nohz） | 影响定时精度与开销 |

### 5.3 构建步骤（x86 示例）

```
# 1. 取源码
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.12.tar.xz
tar xf linux-6.12.tar.xz && cd linux-6.12

# 2. 以当前系统配置为基线，打开 RT
cp /boot/config-$(uname -r) .config
scripts/config --enable  PREEMPT_RT \
               --disable PREEMPT_VOLUNTARY \
               --enable  HIGH_RES_TIMERS \
               --enable  NO_HZ_FULL \
               --disable RT_GROUP_SCHED
make olddefconfig

# 3. 确认开关已生效
grep -E "CONFIG_PREEMPT(_RT)?=|CONFIG_HIGH_RES" .config

# 4. 编译安装
make -j$(nproc)
sudo make modules_install && sudo make install
sudo update-grub && reboot

# 5. 启动后确认
uname -v            # 应带 PREEMPT_RT 字样
cat /sys/kernel/realtime   # 输出 1 表示 RT 已生效
```

交叉编译到 ARM 板卡时，把 `make -j$(nproc)` 换成带 `ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-` 的形式，并用 `make deb-pkg` 生成可安装的 deb 包再拷到板端安装。

# 6　系统级调优

仅仅换成 RT 内核通常还不够——默认配置下依然可能跑出几百微秒的抖动。标准做法是把一个核**彻底让出来**给实时任务。

### 6.1 CPU 隔离

```
# /etc/default/grub 的 GRUB_CMDLINE_LINUX 中追加
isolcpus=2-3 nohz_full=2-3 rcu_nocbs=2-3 irqaffinity=0-1 \
intel_idle.max_cstate=0 processor.max_cstate=1
```

| 参数 | 作用 |
| --- | --- |
| `isolcpus=` | 把指定 CPU 从通用调度器中摘出来，普通任务不会跑上去 |
| `nohz_full=` | 让这些核停掉周期 tick，消除定时器中断干扰 |
| `rcu_nocbs=` | 把 RCU 回调搬到其他核执行，避免在实时核上长时间运行 |
| `irqaffinity=` | 默认中断亲和性，把中断赶出隔离核 |

## 6.2 中断亲和性

```
# 关闭 irqbalance，避免它把中断又挪回隔离核
sudo systemctl stop irqbalance && sudo systemctl disable irqbalance

# 查看中断分布
cat /proc/interrupts

# 把某个中断绑到 0、1 号核（列表形式更直观）
echo 0-1 > /proc/irq/<IRQ号>/smp_affinity_list
```

## 6.3 电源管理与内核噪声

```
# 固定频率策略，避免 DVFS 抖动
sudo cpupower frequency-set -g performance

# 关闭透明大页后台合并
echo never > /sys/kernel/mm/transparent_hugepage/enabled

# 隔离核上的内核看门狗（nohz_full 通常已处理，必要时手动关）
echo 0 > /proc/sys/kernel/watchdog
```

## 6.4 调度与内存

```
# RT 任务默认最多占用 95% CPU，超出会被强制休眠；硬实时场景可关闭保护
# ⚠️ 关闭后 RT 线程一旦死循环，整机将失去响应，务必先配好看门狗
echo -1 > /proc/sys/kernel/sched_rt_runtime_us

# 运行实时任务：绑核 + FIFO 优先级
taskset -c 2 chrt -f 80 ./your_app
```

> **要点**
>
> 调优顺序建议：先测基线（`cyclictest`）→ 再逐项开关隔离参数 → 每次只改一项并重测。一次性全开会导致出问题时无法定位是哪一项带来的改善或恶化。

# 7　应用程序编写准则

## 7.1 优先级设计

| 角色 | 典型优先级 | 说明 |
| --- | --- | --- |
| 内核 IRQ 线程 | FIFO 50（默认） | 可通过 `/proc/irq/N/` 与 `chrt` 调整 |
| 实时主循环 | FIFO 60 ~ 90 | 必须高于它所依赖外设的 IRQ 线程 |
| 数据记录 / 通信 | FIFO 20 ~ 40 | 允许被主循环抢占 |
| 监控看门狗 | FIFO 99 | 只做超时检测，逻辑必须极短 |

## 7.2 实时循环里的禁忌

- **禁止动态分配**：`malloc` / `free` / `new` 可能触发加锁与缺页，全部改为启动时预分配。
- **禁止阻塞式 IO 与打印**：`printf`、`fopen`、`write` 到慢设备都可能阻塞毫秒级。调试信息写入无锁环形缓冲区，由低优先级线程落盘。
- **禁止首次加载的惰性开销**：动态库的符号解析、首次调用的代码页换入，都在启动阶段用"预热调用"完成。
- **禁止无界循环**：所有 `while` 都要有基于时间的退出条件与超时计数。
- **警惕 `std::mutex`**：glibc 的 `std::mutex` 默认不带优先级继承，实时任务之间请用 `PTHREAD_PRIO_INHERIT` 的 `pthread_mutex_t`（C++ 可用 `pthread_mutex_t` 封装成 `std::lock_guard` 的 BasicLockable）。

## 7.3 定时与时间基准

用绝对时间的 `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, ...)`，而不是相对睡眠——相对睡眠会把每次的处理耗时累加进周期，导致长期漂移。需要多路定时时用 `timerfd` 配合 `epoll`。

## 7.4 代码骨架

```
#define _GNU_SOURCE
#include <pthread.h>
#include <sched.h>
#include <sys/mman.h>
#include <time.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdint.h>

static const long PERIOD_NS = 1000000;   /* 1 ms */

static void *rt_loop(void *arg)
{
    struct sched_param sp = { .sched_priority = 80 };
    pthread_setschedparam(pthread_self(), SCHED_FIFO, &sp);

    struct timespec next;
    clock_gettime(CLOCK_MONOTONIC, &next);

    for (;;) {
        /* ---- 实时工作区：只做内存里的确定性计算与外设寄存器访问 ---- */
        /* 禁止 malloc / printf / 文件 IO / 可能阻塞的系统调用              */

        next.tv_nsec += PERIOD_NS;
        while (next.tv_nsec >= 1000000000L) {
            next.tv_nsec -= 1000000000L;
            next.tv_sec++;
        }
        clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next, NULL);
    }
    return NULL;
}

int main(void)
{
    /* 锁住全部内存，杜绝运行期缺页 */
    if (mlockall(MCL_CURRENT | MCL_FUTURE) != 0) { /* 记录告警 */ }

    /* 抑制深层 C-state：这个 fd 必须一直打开，不能 close */
    int lat_fd = open("/dev/cpu_dma_latency", O_WRONLY);
    if (lat_fd >= 0) {
        int32_t zero = 0;
        write(lat_fd, &zero, sizeof(zero));
    }

    pthread_t th;
    pthread_attr_t attr;
    pthread_attr_init(&attr);
    pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);
    pthread_attr_setschedpolicy(&attr, SCHED_FIFO);
```

```
    struct sched_param sp = { .sched_priority = 80 };
    pthread_attr_setschedparam(&attr, &sp);
    pthread_create(&th, &attr, rt_loop, NULL);

    pthread_join(th, NULL);
    return 0;
}
```

# 8 度量与验证

## 8.1 cyclictest

```
sudo apt install rt-tests
cyclictest -m -Sp90 -i200 -h400 -D 10m -t4
```

| 参数 | 含义 |
| --- | --- |
| `-m` | 锁定内存（mlockall） |
| `-S` | 标准化的唤醒时间测量方式 |
| `-p90` | 线程优先级 90 |
| `-i200` | 周期 200 µs |
| `-h400` | 打印到 400 µs 的直方图 |
| `-D 10m` | 运行 10 分钟 |
| `-t4` | 4 个线程，覆盖多核 |

看 **Max** 和直方图最右侧的长尾，不要看 Avg。测试必须在**有负载**的情况下做——空载测出来的数字没有意义。

## 8.2 ftrace 追踪器

```
# 硬件/固件占用了多久（SMI、总线争用）
cd /sys/kernel/tracing
echo hwlat > current_tracer; cat trace

# 系统噪声：有哪些高优先级活动在打断我的核
echo osnoise > current_tracer; cat trace

# 最长的"关中断 / 关抢占"区间及其调用栈
echo preemptirqsoff > current_tracer; cat trace
```

### 8.3 参考量级

| 场景 | Max 延迟 | 说明 |
| --- | --- | --- |
| 普通内核，空载 | 数百 µs ~ 数 ms | 抖动主要来自驱动与中断 |
| 普通内核，有负载 | 数 ms ~ 数十 ms | 长尾随负载显著恶化 |
| RT 内核，未调优 | 200 ~ 800 µs | 受 C-state、RCU、tick 影响 |
| RT 内核，隔离核调优（x86） | 20 ~ 60 µs | 接近该硬件的理论上限 |
| RT 内核，隔离核调优（ARM SoC） | 100 ~ 300 µs | 固件、DDR 争用、缓存一致性开销更大 |

# 9 代价、边界与替代方案

## 9.1 代价

- **吞吐下降。**线程化中断与睡眠锁带来更多上下文切换，典型损耗 3% ~ 10%，高并发 IO 场景更明显。
- **驱动兼容性。**厂商闭源驱动里常有长临界区甚至违反原子上下文规则，图形、VPU、摄像头链路尤其明显。
- **调试复杂度上升。**同一段代码在 RT 与非 RT 下的行为差异需要额外验证。
- **维护成本。**自维护内核意味着要跟进安全补丁与硬件支持。

## 9.2 何时不该用

Web 服务、批处理、大数据、机器学习训练——这些场景要的是吞吐，RT 只会让它们更慢。选型准则只有一句：**只有当你能写出"最坏响应时间必须小于 X 微秒，并且可以被反复验证"这句话时，才值得上 RT。**否则 lowlatency 内核 + 合理调优就够了。

## 9.3 替代方案

| 方案 | 原理 | 延迟量级 | 代价 |
| --- | --- | --- | --- |
| PREEMPT_RT | 单内核，全面可抢占 | 数十 µs | 吞吐略降，生态成熟 |
| Xenomai / EVL | 双内核，实时核接管中断 | 数 µs | 树外维护，驱动需适配 |
| Jailhouse | 硬件分区，把核静态分给实时域 | 取决于分区内系统 | 资源无法共享，配置复杂 |
| AMP（异构核） | 实时任务跑在 MCU / Cortex-R 核上 | 亚 µs 级 | 需硬件支持，跨核通信成本 |

# 10　落地示例：RK3588 上的取舍

手势识别这类视觉应用本身**不需要** RT：30 fps 采集对 5 ms 的调度抖动毫无感知，把时间花在 RT 上是过度设计。真正需要 RT 的是下面这类需求：

- **多传感器硬同步。**摄像头 + IMU 做动作捕捉时，抖动会直接变成数据错位，必须用 `CLOCK_MONOTONIC` 打时间戳并保证采集线程的唤醒抖动可知。
- **闭环控制。**视觉伺服、云台跟踪等需要固定周期输出，周期抖动直接影响控制稳定性。
- **确定性 IO。**与下位机走 CAN / EtherCAT 时，通信周期不能被内核噪声打乱。

RK3588 是 4×Cortex-A76 + 4×Cortex-A55 的大小核结构，很自然的切分是：

```
# A55（0-3）：系统、网络、日志、图形
# A76（4-7）：实时任务，其中 6-7 号核完全隔离
isolcpus=6-7 nohz_full=6-7 rcu_nocbs=6-7 irqaffinity=0-3

taskset -c 6 chrt -f 80 ./capture_app
```

> **注意**
>
> Rockchip BSP 内核（如 5.10）的 RT 支持需要厂商提供对应补丁，主线 6.12+ 虽已内置 PREEMPT_RT，但外设驱动（ISP、VPU、NPU）在主线上的成熟度通常不如 BSP。两条路的取舍要先做验证：**要实时还是要外设。**

# 附录 A　术语表

| 术语 | 含义 |
| --- | --- |
| 硬实时 / 软实时 | 硬实时要求绝不可错过截止时间；软实时允许偶尔超时，只要统计上可接受 |
| 抖动（jitter） | 实际响应时间相对期望值的离散程度，实时系统的核心指标 |
| 原子上下文 | 不能睡眠的执行环境：中断处理、`preempt_disable()` 区间、持 `raw_spinlock` 时 |
| 优先级反转 | 高优先级任务被低优先级任务持有的锁阻塞，而持锁者又被中优先级任务抢占 |
| 优先级继承 | 持锁者临时继承等待者的优先级，从而尽快释放锁，是反转的解决方案 |
| 线程化中断 | 中断处理程序在内核线程中执行，可被高优先级任务抢占 |
| tickless | 空闲或隔离的 CPU 停掉周期性时钟中断，减少无谓唤醒 |
| PI futex | 支持优先级继承的用户空间快速互斥量，需 `PTHREAD_PRIO_INHERIT` |

# 附录 B　命令速查

```
# 确认当前内核是否为 RT
uname -v && cat /sys/kernel/realtime

# 查看 RT 调度带宽限制
cat /proc/sys/kernel/sched_rt_runtime_us

# 查看某进程的调度策略与优先级
chrt -p <pid>

# 临时把进程改为 SCHED_FIFO 优先级 80
chrt -f -p 80 <pid>

# 查看中断在各核上的分布
cat /proc/interrupts | head -20

# 查看 CPU 频率与 governor
cpupower frequency-info

# 查看温度（防止热降频导致抖动）
cat /sys/class/thermal/thermal_zone*/temp

# 延迟基准测试
cyclictest -m -Sp90 -i200 -h400 -D 10m -t4
```